

LECTURE 18

Problem-Solving Workshop: Scheduling and Synchronization

Four fully worked problems for exam practice

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- Practice applying two CPU scheduling algorithms to the same input and comparing results
- Practice tracing a software mutual-exclusion solution through a specific interleaving
- Practice diagnosing a semaphore ordering bug that causes deadlock, not just a wrong answer
- Reinforce why a many-to-one threading model cannot deliver parallelism

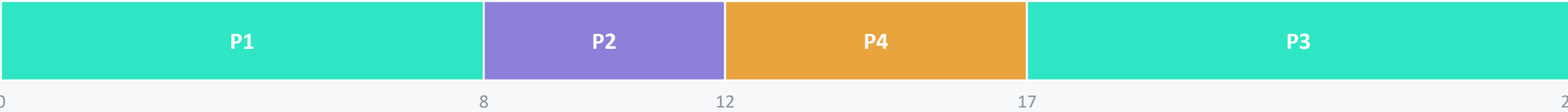
PROBLEM 1

Comparing SJF and Round Robin

Four processes arrive at a single-CPU system as follows. Compute the Gantt chart, waiting time, and turnaround time for (a) non-preemptive SJF and (b) Round Robin with time quantum $q = 4$. Then compare the two results.

Process	Arrival	Burst
P1	0	8
P2	1	4
P3	2	9
P4	3	5

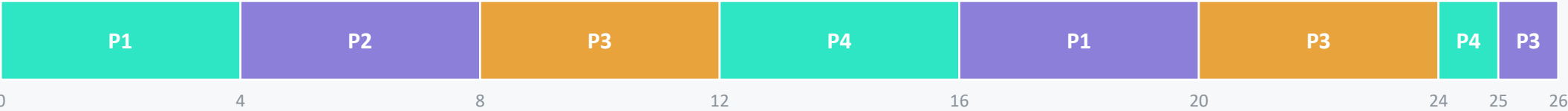
Non-Preemptive SJF



Process	Completion	Turnaround	Waiting
P1	8	$8 - 0 = 8$	$8 - 8 = 0$
P2	12	$12 - 1 = 11$	$11 - 4 = 7$
P4	17	$17 - 3 = 14$	$14 - 5 = 9$
P3	26	$26 - 2 = 24$	$24 - 9 = 15$

Average waiting time = $(0 + 7 + 9 + 15) / 4 = 7.75$. Average turnaround time = $(8 + 11 + 14 + 24) / 4 = 14.25$.

Round Robin, q = 4



Process	Completion	Turnaround	Waiting
P1	20	$20 - 0 = 20$	$20 - 8 = 12$
P2	8	$8 - 1 = 7$	$7 - 4 = 3$
P3	26	$26 - 2 = 24$	$24 - 9 = 15$
P4	25	$25 - 3 = 22$	$22 - 5 = 17$

Avg waiting = 11.75, avg turnaround = 18.25. SJF wins on both averages, but RR gives every process its first CPU slice within one quantum — lower average wait and responsiveness are different goals.

Tracing Peterson's Solution

Two processes, P0 and P1, share flag[0], flag[1] (both initially false) and turn. Each process i (j = the other) runs the code below. Trace the interleaving and determine whether mutual exclusion holds.

```
flag[i] = true;
turn = j;
while (flag[j] && turn == j) { /* busy-wait */ }
    // critical section
flag[i] = false;
```

- 1. P0 executes flag[0] = true 2. P1 executes flag[1] = true
- 3. P0 executes turn = 1 4. P1 executes turn = 0
- 5. P0 evaluates its while condition 6. P1 evaluates its while condition
- 7. P0 enters/finishes critical section, executes flag[0] = false 8. P1 re-evaluates its while condition

Step-by-Step Trace

Step	Action	flag[0]	flag[1]	turn	Effect
1	P0: flag[0] = true	true	false	—	P0 declares intent to enter
2	P1: flag[1] = true	true	true	—	P1 declares intent to enter
3	P0: turn = 1	true	true	1	P0 yields priority to P1
4	P1: turn = 0	true	true	0	P1's write happens after P0's — turn ends up 0
5	P0 checks flag[1] && turn==1	true	true	0	turn==1 is false → P0 does not wait
6	P1 checks flag[0] && turn==0	true	true	0	Both true → P1 busy-waits
7	P0 runs CS, then flag[0]=false	false	true	0	P0's turn in the CS is over
8	P1 re-checks flag[0] && turn==0	false	true	0	flag[0] false → P1 exits loop, enters

Mutual exclusion holds. The decisive moment is step 4: only the last write to the shared variable turn survives — here, P1's write of 0 — which is why P0 proceeds and P1 correctly waits.

PROBLEM 3

Fixing a Buggy Bounded Buffer

A bounded buffer of N slots is protected by three semaphores: mutex (binary, initially 1), empty (counting, initially N), full (counting, initially 0). Find the bug in the producer below, and explain why it causes deadlock.

```
// Buggy producer
wait(mutex);
wait(empty);
    ... add item to buffer ...
signal(full);
signal(mutex);
```

Corrected Order, and Why It Matters

```
// Corrected producer
wait(empty);
wait(mutex);
    ... add item to buffer ...
signal(mutex);
signal(full);
```

The bug is the order of the two `wait()` calls. If the buffer is full (`empty == 0`), the producer acquires `mutex`, then blocks on `wait(empty)` — while still holding `mutex`. The consumer needs `mutex` to remove an item and `signal(empty)`, so it blocks too. Each process holds what the other needs: a genuine circular-wait deadlock, not just a race condition.

WARNING

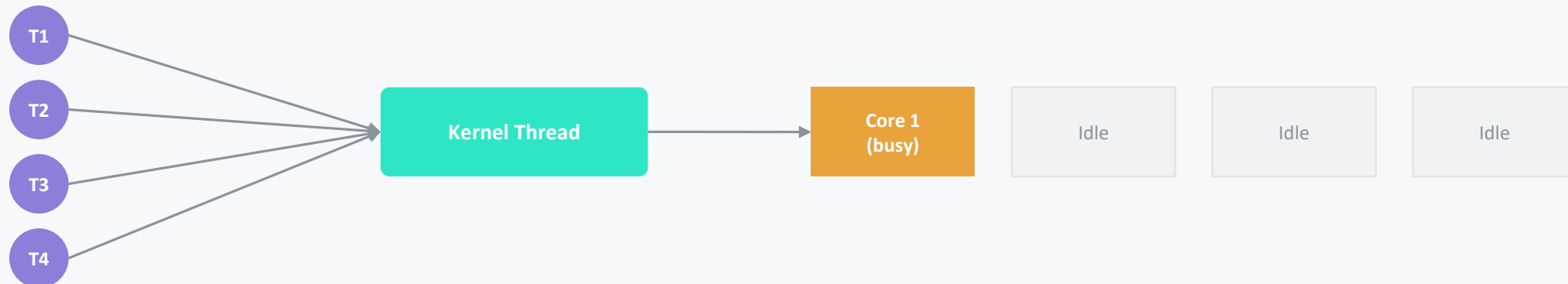
Always acquire a semaphore that might block indefinitely (`empty`, `full`) before the mutual-exclusion lock (`mutex`) another process needs to unblock you.

Many-to-One Threading and Parallelism

Explain why a many-to-one threading model cannot achieve parallelism on a multicore system, even with thousands of user-level threads.

Solution

The thread library multiplexes every user-level thread onto exactly one kernel-level thread. The kernel only ever schedules that single entity, so no matter how many user threads exist, at most one core can be in use at any instant — the rest sit idle. This is also why one blocking system call freezes every thread in the process: the one kernel thread they all depend on is now blocked too.



KEY TAKEAWAYS

Four Problems, Four Lessons

1

SJF minimizes average waiting time; Round Robin maximizes fairness and responsiveness — neither property implies the other

2

Tracing Peterson's solution statement-by-statement is the reliable way to verify mutual exclusion — the shared turn variable's last writer decides

3

A semaphore-ordering bug (mutex before empty/full) causes deadlock, because it lets a process sleep while still holding the lock its unblocker needs

4

A many-to-one threading model's ceiling is fixed by the kernel's view: one schedulable entity means at most one core in use, regardless of thread count

Lecture 19 names the pattern from Problem 3 formally: a circular wait where each process holds something the other needs.